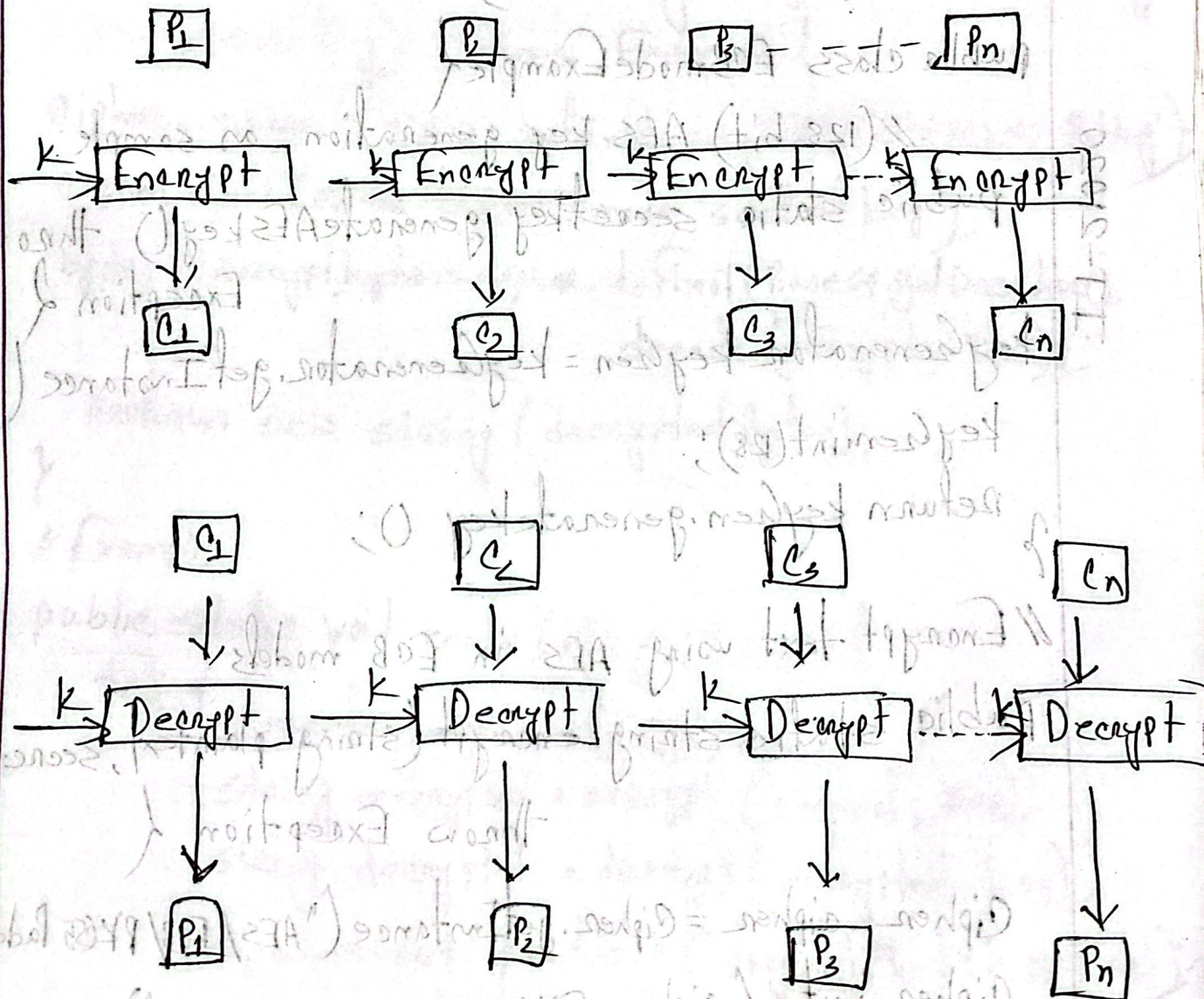


Assignment

ECB (Electronic Codebook): In an, electronic codebook each block of bits of plain text is encoded indepedently with the same key.

Block Diagram:



Java Implementation

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;

public class ECBmodeExample {
    // (128 bit) AES key generation as sample
    public static SecretKey generateAESkey() throws Exception {
        KeyGenerator keyGen = KeyGenerator.getInstance("AES");
        keyGen.init(128);
        return keyGen.generateKey();
    }

    // Encrypt text using AES in ECB mode
    public static String encrypt(String plaintext, SecretKey key)
        throws Exception {
        Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
        cipher.init(Cipher.ENCRYPT_MODE, key);
    }
}
```



```
byte[] encryptBytes = cipher.doFinal(Plaintext.getBytes());  
return Base64.getEncoder().encodeToString(encryptBytes);
```

}

// Decrypt cipher text using AES in ECB ~~using~~ mode

```
public static String decrypt(String encryptedText, SecretKey  
secretKey) throws Exception {
```

```
    Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
```

```
    cipher.init(cipher.DECRYPT_mode, secretKey);
```

```
    byte[] decryptBytes = cipher.doFinal(Base64.getDecoder().  
        decode(encryptedText));
```

```
    return new String(decryptBytes);
```

}

// Example

```
public static void main(String[] args) {
```

```
    try { SecretKey = generateAESkey();
```

```
        String encrypted = encrypt(original, key);
```

```
        String decrypted = decrypt(encrypted, key);
```

```
        System.out.println("original Text: " + original);
```

```
        System.out.println("Encrypted Text: " + encrypted);
```

```
        System.out.println("Decrypted Text: " + decrypted);
```

}


```
try {
    // ...
} catch (Exception e) {
    e.printStackTrace();
}
```

output

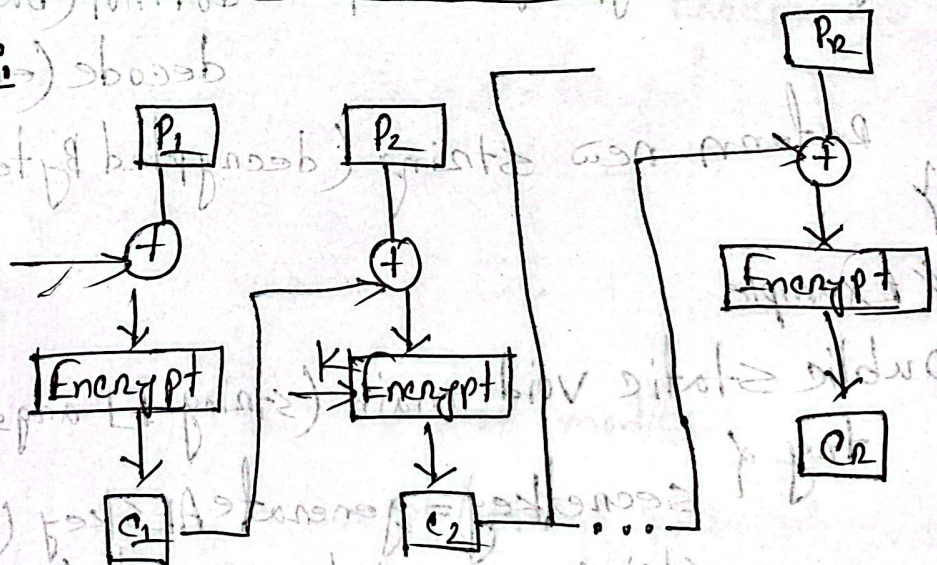
Original Text: Hello cyber world;

Encrypted Text: fRQ+HykBP3AseX1v91Qw119nwpgH

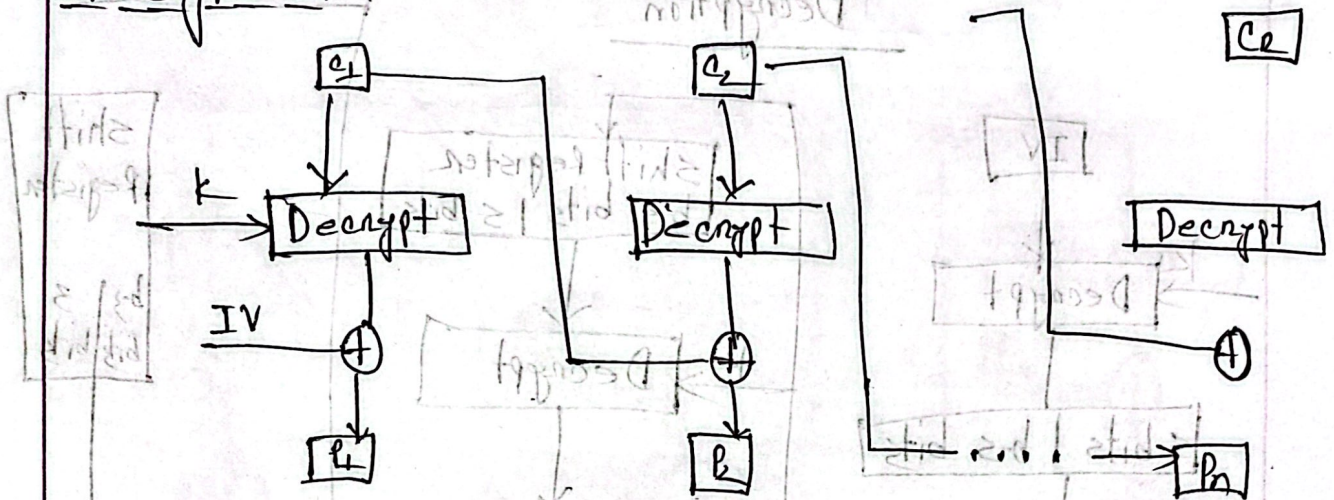
Decrypted Text: Hello cyber world!

CBC (Cipher Block Chaining)

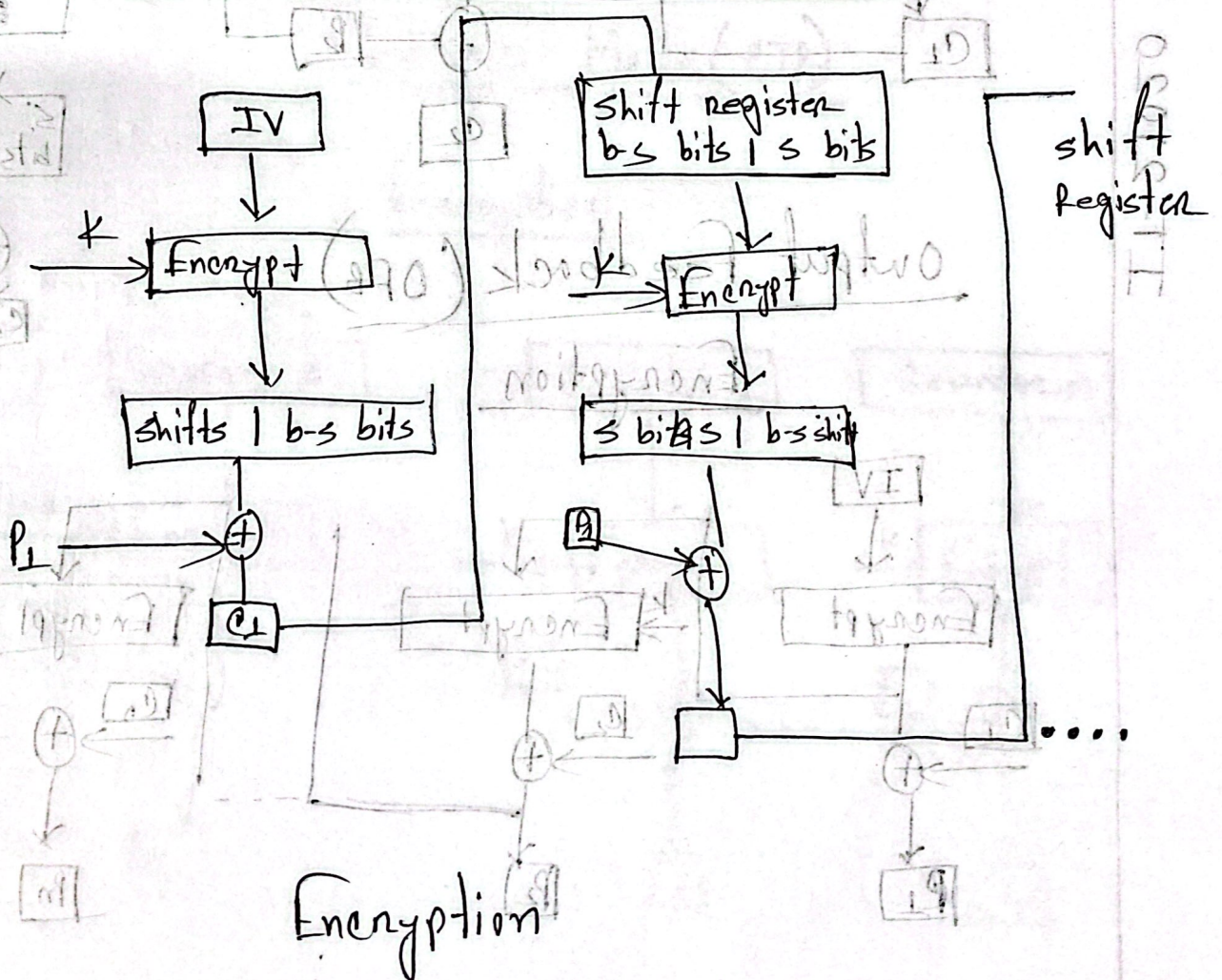
Encryption:



Decryption:

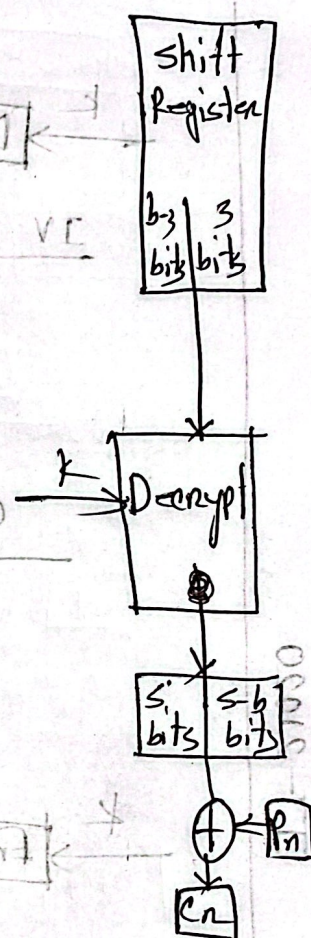
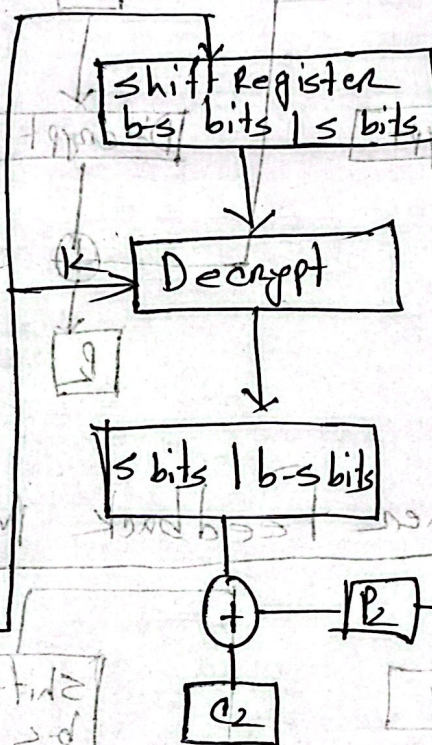
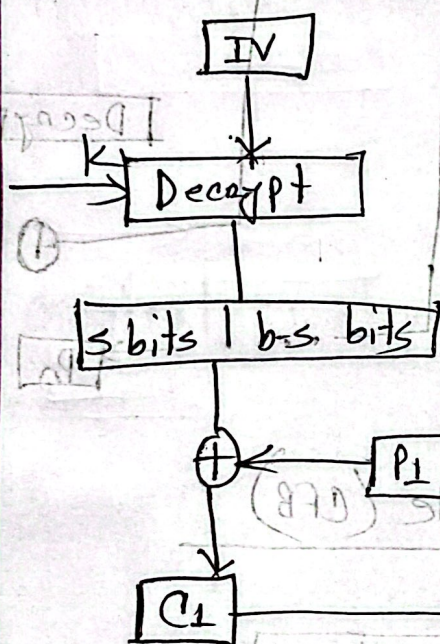


Cipher Feedback Mode (CFB)



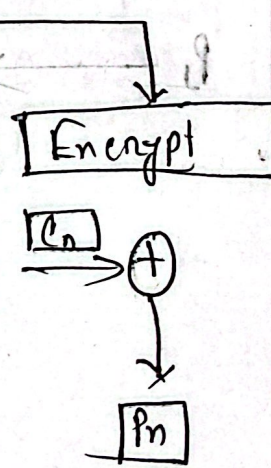
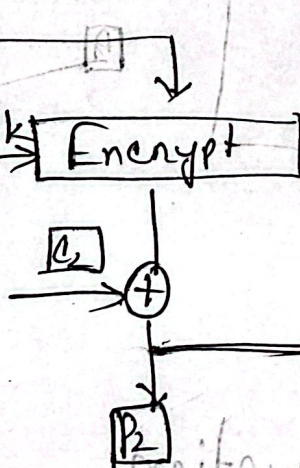
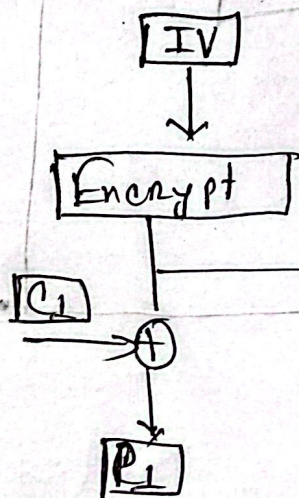
Encryption

Decryption

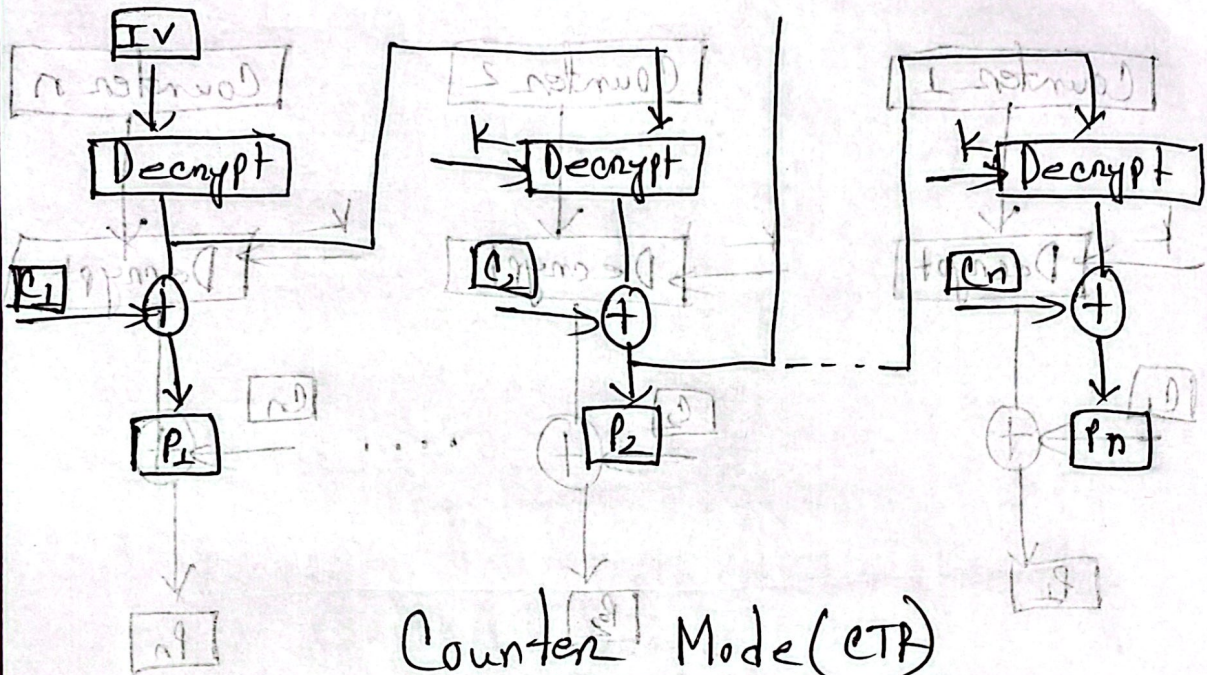


Output Feedback (OFB)

Encryption

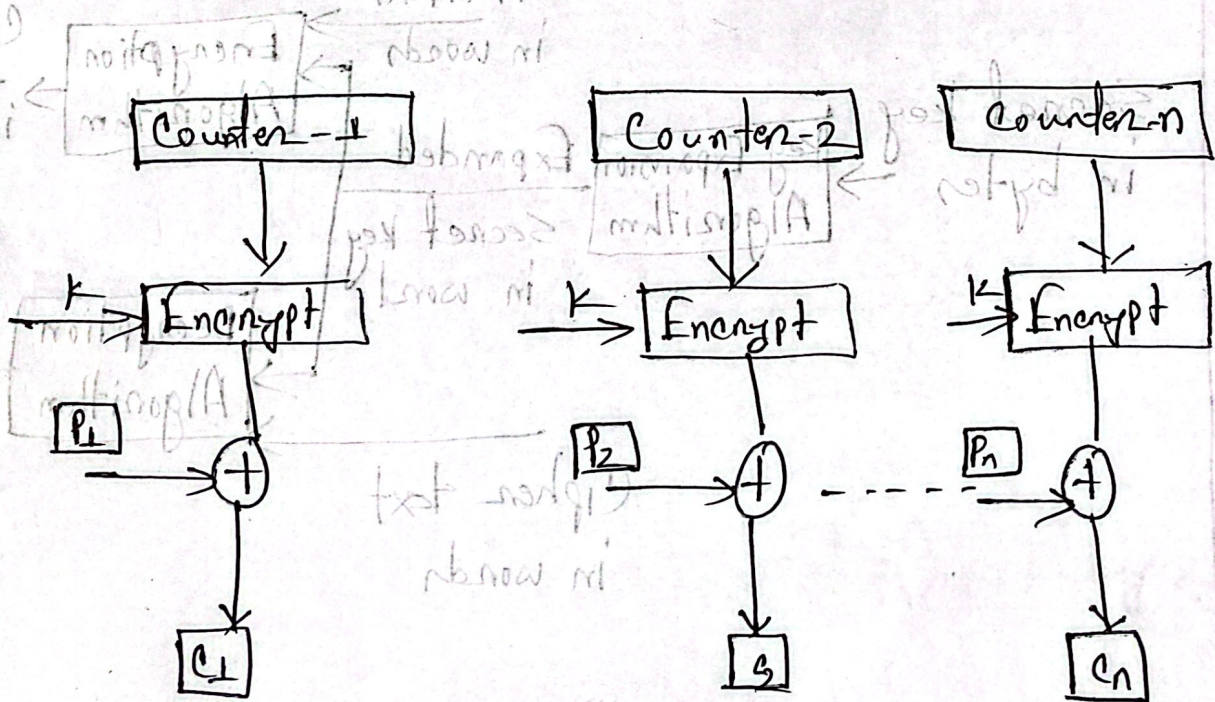


Decryption

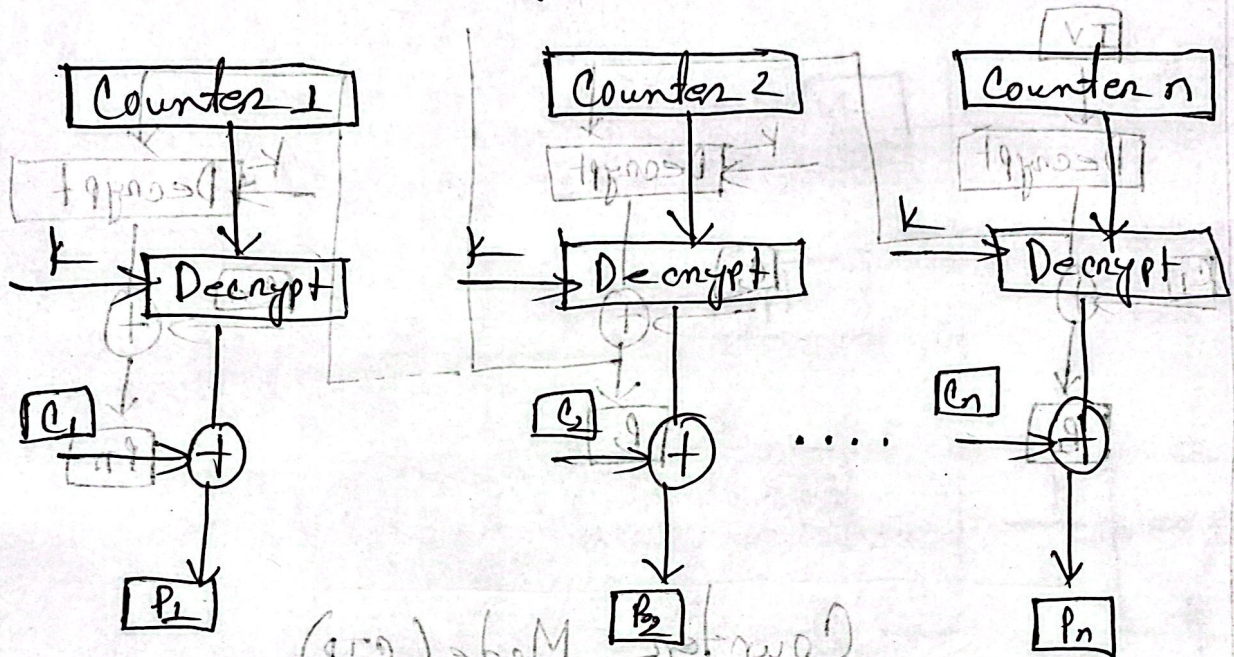


Counter Mode (CTR)

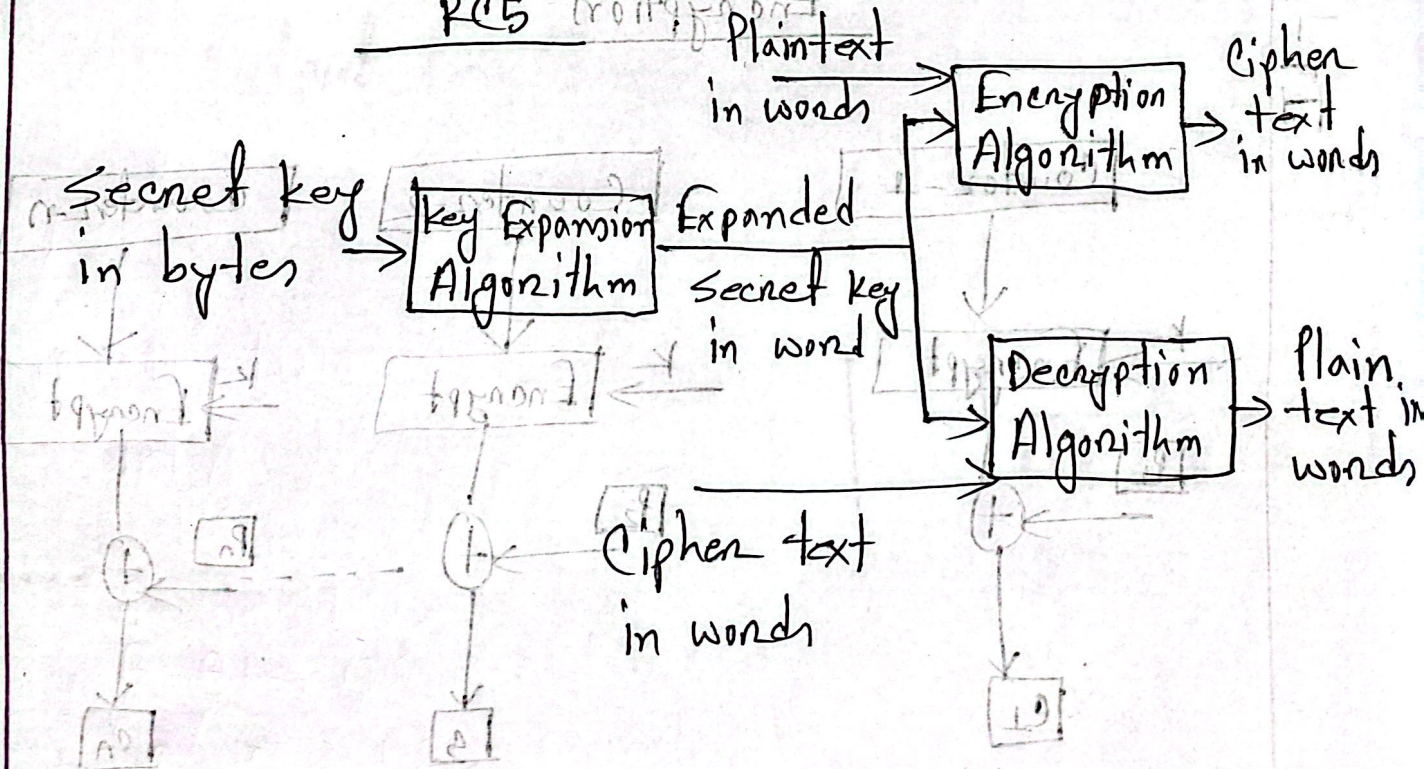
Encryption



Decryption



PC5



code:

```
Package org.example.net;
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
import java.nio.ByteBuffer;
import java.nio.channels.StandardCharsets;
import java.util.Arrays;
```

```
Public class PC5x1 extends Application {
    Private static final int p = 0XB7E25163;
    Private static final int Q = 0x9E3779B9;
```

```
Private static int rot1(int x, int y) {
    Return (x < y) ? (x >> (w - y)) :
```

```
Private static void reskeysetup(byte[] key, int[] L) {
    int[] L = new int[L.length];
    for (int i = B-1; i >= 0; i--) {
        L[i] = (L[i] << 8) + key[i] & 0xFF);
```


$s[0] = P;$

for (int i=1; i < 2*(P+1); i++) {

$s[i] = s[i-1] + Q;$

int A=0, B=0;

int i=0; j=0;

for (int k=0; k < 3 * Math.Max(2*(P+1), C); k++) {

$A = s[i] = \text{rot}1(s[i] + A + B, 3);$

$B = L[j] = \text{rot}1(L[j] + A + B, (A+B) \% W, i = (i+1) \% (2*(P+1)));$

$j = (j+1) \% C;$

private static void RES Encrypt (int[] s, int[] data) {

 int A = data[0];

 int B = data[1];

$A = A + s[0];$

$B = B + s[1];$

 for (int i=1; i <= R; i++) {

$A = \text{rot}1(A \wedge B, B) + s[2*i];$

$B = \text{rot}1(B \wedge A, A) + s[2*i+1];$

res keysetup (key s);

byte[] PlainBytes = PlainText.getBytes (standard channels UTF8)

int paddedLength = (PlainBytes.Length + 7) / 8 * 8;

byte[] Padded = Array.Copy of (PlainBytes, PaddedLength);

StringBuffer ciphertext = new StringBuffer()

ByteBuffer buffer = ByteBuffer.wrap (Padded);

while (buffer.hasRemaining())

int A = buffer.getInt();

int B = buffer.getInt();

int[] data = { A, B }

res Encrypt (s, data);

ciphertext.append (string.format ("%08x %08x" data[0], data[1]));

return cipher.toString().trim();

@Override

```
public void start(Stage stage) {  
    TextField input = new TextField();  
    input.setPromptText("Enter English word to encrypt");  
    Button encryptButton = new Button("Encrypt");  
    Text Area output = new Text Area();  
    output.setEditable(false);  
    output.setWrapText(true);  
    encryptButton.setOnAction(e -> {  
        String plaintext = input.getText();  
        if (plaintext != null && !plaintext.isEmpty()) {  
            String ciphertext = encryptText(plaintext);  
            output.setText(ciphertext);  
        }  
        else {  
            output.setText("please enter some text");  
        }  
    });  
}
```

IT-21020

IT-21020


```
vBox.layout = new VBox(10, inputEncrypButton, output);
layout.setStyle("-fx-padding: 20");
Scene scene = new Scene(layout, 500, 300);
stage.setTitle("AES Encryption Tool");
stage.setScene(scene);
stage.show();
```

```
public static void main(String[] args) {
    launch(args);
}
```

Output:

Input: hello world

Padding into 64 bit blocks

Block 1: "hello wo" (8 bytes)

Block 2: "rd 10101010" (padded)

Encrypted output:

07c9f2a2 b1557b56 2493583d 5f0268b3

original: hello world